

Module Five Journal

Courtney Jenkins

Southern New Hampshire University

CS 320: Software Test & Automation

Sherif Antoun

June 21, 2026

The application consisted of three object classes and their associated service class. Each object class used specific requirements determined by the customer to enforce specific validation logic, while the service classes provided the functions to add, delete, and update the objects. A single UI was created that tied all six classes together to create a singular application. My job was to determine, develop, and implement testing to ensure that the application would meet the business requirements and provide reliable service. Determining the tests needed for the software came from examining the customer requirements for the project. I was given specific requirements for each object class field to not be null, empty, or only whitespace. All object class fields had specified character lengths, and some had specific requirements (e.g., phone is only digits, appointmentDate cannot be in the past). A logical testing approach was needed to ensure both valid and invalid input conditions and field integrity for each object (Garcia, 2017). From the same customer requirements, the respective service requirements iterated the need to add, delete, and update specific fields. With all these requirements in mind, I utilized the following tests: positive testing for input and method execution validation, negative testing for input converse to the customer requirements, boundary testing for field requirements, and exception testing for improper field input (Garcia, 2017). Once determined, I developed my JUnit tests and documented them in the Arrange-Act-Assert format. This format clearly delineates test setup,

actions being tested, and the verification of the outcomes for each test (Garcia, 2017). I ensured that my tests could be easily maintained and that there is strong, purposeful traceability with their implementation. I determined that I've implemented high-quality tests based on analyzing the customer requirements. My tests included input validation that takes into account successful and unsuccessful efforts to ensure that all conditions are evaluated. I used boundary testing to verify that the logic for each field meets the specification and behavior is consistent at both maximum and minimum ranges. My tests ensure that data is preserved after a failure condition, showing that the logic rejects bad inputs and maintains the data's integrity. Within the service classes, I use the `@BeforeEach` annotation to create a new instance of the service prior to each test's execution. Not only does this ensure that my tests are repeatable, but it reduces the chances of false positives and false negatives (Garcia, 2017). I would estimate that the test coverage is <90%. A high test coverage does not necessarily translate to a useful, efficient test coverage (Garcia, 2017). Based on my tests covering all constructors, setters, validation paths, exception paths, and service methods, I determine that I have implemented a highly efficient test program.

Technical soundness of my code was achieved by design and implementation based around requirements, test isolation, independent test execution, specific forced behavior, and defect testing behaviors. A good example would be Figure 1 below. Here, I utilized the requirement - cannot be null and no longer than 30 characters- to generate tests to enforce this logic. I determined that an empty entry and a whitespace-only entry did not meet the customer's expectations, as well. My tests forced these conditions to show that the code's logic rejected these states and preserved the condition of the field after the failure occurred. The `@Test` annotation tells the program to run this test each time the JUnit test script is executed. I used a fresh testing instance of `testContact()` each time for test isolation and repeatability. `assertThrows`

is used to catch the expected error and return the result for a satisfactory test. The assertEquals is used to verify the correct value is maintained for the field after the error.

```
@Test
void testSetAddress_Null_ThrowsException() {
    Contact c = testContact();
    assertThrows(IllegalArgumentException.class, () -> c.setAddress(null));
}

@Test
void testSetAddress_IsEmpty_ThrowsException() {
    Contact c = testContact();
    assertThrows(IllegalArgumentException.class, () -> c.setAddress(""));
}

@Test
void testSetAddress_WhitespaceOnly_ThrowsException() {
    Contact c = testContact();
    assertThrows(IllegalArgumentException.class, () -> c.setAddress(" "));
}

@Test
void testSetAddress_TooLong_ThrowsException() {
    Contact c = testContact();
    assertThrows(IllegalArgumentException.class, () -> c.setAddress("420 Error Ave, Debugger City, CS 00320"));
}

@Test
void testSetAddress_PreservedAfterNullUpdate() {
    Contact c = testContact();
    assertThrows(IllegalArgumentException.class, () -> c.setAddress(null));
    assertEquals("101 Main St", c.getAddress());
}

@Test
void testSetAddress_PreservedAfterIsEmptyUpdate() {
    Contact c = testContact();
    assertThrows(IllegalArgumentException.class, () -> c.setAddress(""));
    assertEquals("101 Main St", c.getAddress());
}

@Test
void testSetAddress_PreservedAfterWhitespaceOnlyUpdate() {
    Contact c = testContact();
    assertThrows(IllegalArgumentException.class, () -> c.setAddress(" "));
    assertEquals("101 Main St", c.getAddress());
}

@Test
void testSetAddress_PreservedAfterTooLongUpdate() {
    Contact c = testContact();
    assertThrows(IllegalArgumentException.class, () -> c.setAddress("420 Error Ave, Debugger City, CS 00320"));
    assertEquals("101 Main St", c.getAddress());
}
```

Figure 1: JUnit test case for setAddress() method

Referencing Figure 1, my testing efficiency is maintained in the highest state because of the maintainability, readability, and reduced redundancy that can be observed in the JUnit test cases (Garcia, 2017).

In the project, I used a variety of testing techniques, and some I would like to focus on are boundary value analysis, negative testing, and exception testing. Boundary value analysis is critical to the execution of these tests due to constraints placed on the field character counts (e.g., testing IDs cannot be < 10 characters, task description cannot be < 50 characters, phone must be exactly 10 digits). I fulfilled these requirements by testing constructors and setters over the character limit. In a practical sense, boundary values are some of the most common input errors that can occur with users. My boundary value tests help prevent validation failures, data truncation, and unexpected system response (Garcia, 2017). These errors can result in corruption

of the databases and buffer overflows that might result in the application crashing. Negative testing helps reinforce the theme of input validation by ensuring that user input meets the application's requirements (e.g., null first names, phone numbers with non-digits, appointment dates occurring in the past). These tests were conducted with constructors and setters made with blatant input failures that correspond to an `assertThrows` to determine the efficacy of the JUnit test case. Not only did I ensure that the character limits were achieved during user input, but I tested strict adherence to specific input validation points. These tests ensure safe and reliable responses from the application with user interaction. A potential fallout from improper input validation could be violating the integrity of the application's security and interrupting the UX (Garcia, 2017). Exception testing allowed me to verify that the application would generate expected responses in the face of invalid operations (e.g., attempting to add a duplicate ID, attempting to delete a non-existent object, attempting to update with invalid data). Robust exception handling tests validate the protection and proper function with unexpected inputs or operations (Garcia, 2017). One testing technique that I did not employ in my application is the `@ParameterizedTest` annotation. This type of testing is useful for large projects with hundreds of similar test cases and allows you to test logic with multiple input values (e.g., multiple invalid contact IDs, multiple address length combinations) (Garcia, 2017). It helps to reduce duplicate code and improves the overall maintainability of the test cases. Another example of a testing technique not used that could benefit would be integration testing. My tests focused on unit testing for a singular class, but integration testing validates the interrelation between all the components (Garcia, 2017). Integration testing would take place to validate the UI and UX of my application, and help ensure proper operation after all modules are combined. These tests often use external testing environments to mirror real-world use of the application (Obergon, 2023).

2023).

During the course of this project, I adopted a cautious and thorough mindset to ensure I met critical milestones for each project. Testing isn't just hammering away at every nail head - it's the thoughtful strike that meets the mark to ensure it's driven down in success. The complexity between each object and its related service can be convoluted, but testing helps encapsulate each interaction to a definable point. For instance, each object is instantiated with specific, rigorous logic that needs to be malleable enough to change without error by a service class. They constantly reference each other, but remain independent in their operation. Complexity, such as this, can increase exponentially with the breadth of the code. There exists, in of as much, a strong bias that the testing cases constructed are satisfactory. It is the responsibility of the software tester to ensure that they are meeting the requirements of the customer and anticipating the interaction with the user. A good example would be the first interaction of my testing cases. I constructed test cases for null conditions, but did not anticipate empty or whitespace entries for fields. These could cascade into issues with the UX, UI, or an external database (if employed). My duty as a software tester has evolved to add in my development process leading questions, so as to flesh out the full view of the software project. Efficient and thorough testing always leads to a positive interaction with the end application and helps with maintaining the project during the maintenance phase of the SLDC. Referencing my previous example, I was provided feedback to include these tests and push the envelope and construct tests to ensure valid data was being maintained after error handling. This is not a concept I had fully realized going into test design, and I am sure to consider this with future projects as a software engineering professional.

References

GarcíaB. (2017). *Mastering software testing with JUnit 5 : comprehensive guide to develop high quality Java applications*. Packt Publishing.

Obergon, A. (2023, March 29). *Effective Java Testing Strategies — Unit, Integration, and End-to-End Testing Techniques Overview*. Medium.

<https://www.medium.com/@AlexanderObergon/effective-java-testing-strategies-unit-integration-and-end-to-end-testing-techniques-5ade73ab259a>